

Goxpyriment: A Go Framework for Behavioral and Cognitive Experiments

Christophe Pallier^{1,2*}, Julie Bonnaire² and Marie-France Fourcade²

¹EMR 9003, Centre National de la Recherche Scientifique,
Gif-sur-Yvette, France's.

²ISNERM-CEA Cognitive Neuroimaging Unit, NeuroSpin,
Gif-sur-Yvette, France.

*Corresponding author(s). E-mail(s): christophe@pallier.org;

Abstract

We describe **goxpyriment**, an open-source software framework for programming behavioral and cognitive experiments in the Go programming language. The library wraps the SDL3 graphics library to provide hardware-accelerated, VSYNC-locked stimulus presentation and is designed to address two practical limitations of Python-based experiment tools: runtime environment complexity that complicates deployment across laboratory computers, and unpredictable pause times introduced by Python's garbage collector during timing-sensitive sections. Go is a compiled language whose garbage collector can be disabled programmatically; combined with VSYNC synchronization, this allows stimulus onsets and offsets to be aligned to individual display frames. **goxpyriment** exposes a layered package architecture — **control** (experiment lifecycle), **stimuli** (visual and audio objects), **io** (window, keyboard, mouse, data files), **design** (trial and block structure), **clock** (timing utilities), and **triggers** (hardware synchronization with EEG/MEG recording systems). The visual stimulus set includes text, geometric shapes, images, Gabor patches, drifting sinusoidal gratings, random-dot motion clouds, random-dot stereograms, and video sequences. Audio support covers WAV playback, procedural tone generation, and time-windowed segment extraction with linear ramps. High-precision rapid serial visual presentation is handled by dedicated stream functions that disable garbage collection and log predicted and actual stimulus onsets. An experiment compiles to a single self-contained binary

with no runtime dependencies, simplifying distribution to collaborators and participant testing computers. The framework is released under the GNU General Public License v3 and available at <https://github.com/chrplr/goxpyriment>.

Keywords: experiment software, stimulus presentation, timing precision, Go, SDL3, behavioral experiments

Introduction

Experiment-control software occupies a central position in behavioral and cognitive neuroscience: the temporal precision of stimulus delivery and response recording, the ergonomics of experiment programming, and the ease of deploying experiments to laboratory hardware each affect the quality and reproducibility of empirical data. Over the past two decades, Python has become the dominant language for experiment programming, primarily through two libraries: Epyriment (Krause & Lindemann, 2014) and PsychoPy (Peirce et al., 2019). Before Python, MATLAB with the Psychophysics Toolbox (Brainard, 1997; Kleiner et al., 2007) was the de facto standard in many laboratories. More recently, web-based frameworks such as jsPsych (de Leeuw, 2015; de Leeuw et al., 2023) and lab.js (Henninger et al., 2022) have enabled large-scale online data collection.

Each class of tool involves tradeoffs. MATLAB offers reliable timing on well-configured hardware but is proprietary, expensive, and increasingly difficult to maintain across operating-system updates. Python tools provide an open, flexible programming environment with a large scientific ecosystem, but they introduce two practical difficulties. First, deploying an experiment to a computer that does not already have a compatible Python environment requires package management (conda, pip, venv) and is a frequent source of breakage across operating-system versions, hardware architectures, and time. Second, Python’s garbage collector can introduce unpredictable pause times that corrupt the timing of stimulus presentation and response recording during the critical trial loop. While techniques exist to mitigate this — pre-allocating stimulus objects, using NumPy arrays to avoid Python-level allocation, or timing the experiment around known GC cycles — they add complexity and are not always sufficient. Web-based tools trade timing precision for accessibility: browser-mediated stimulus presentation on general-purpose operating systems is subject to event-loop latency, compositor delays, and monitor polling intervals that make sub-frame timing difficult or impossible (Bridges et al., 2020; Anwyl-Irvine et al., 2021).

The Go programming language (Donovan & Kernighan, 2016) offers a different point in the design space. Go compiles to native machine code for each target platform and produces self-contained binaries with no runtime dependencies. Its garbage collector is concurrent and can be suspended for the duration of a timing-critical section by calling `debug.SetGCPercent(-1)`, after which it is restored without data loss. The language provides strong static typing, explicit error handling, and a straightforward concurrency model. These properties make it an attractive substrate for a behavioral

experiment library that prioritizes timing reliability and deployment simplicity over flexibility in the scientific computing ecosystem.

`goxpyriment` is a Go library that provides a high-level API for behavioral and cognitive experiments, deliberately modeled on Expyriment’s design philosophy: an `Experiment` object owns all resources; stimuli are presented by calling `exp.Show(stim)`; responses are collected by calling `exp.Keyboard.WaitKeysRT(keys, timeout)`; and data are written to a CSV-like output file whose metadata header records the experimental context. The library uses SDL3 (Simple DirectMedia Layer, version 3) via the `go-sdl3` bindings (Zyko0, 2024) for hardware-accelerated 2D rendering and event handling. The SDL3 renderer presents frames synchronously with the monitor’s vertical retrace signal (VSYNC), ensuring that each stimulus onset and offset coincides with a frame boundary.

The present article describes the design and capabilities of `goxpyriment`, discusses its timing architecture, and situates it relative to existing tools.

Software Design and Architecture

Package Structure

`goxpyriment` is organized as a Go module with a layered package hierarchy. Packages at lower levels are independent of packages above them, which keeps the dependency graph acyclic and allows individual packages to be understood and tested in isolation.

<code>control/</code>	experiment lifecycle and orchestration
<code>stimuli/</code>	visual and audio stimulus objects
<code>io/</code>	SDL window/renderer, keyboard, mouse, data files
<code>design/</code>	trial/block structure and randomization
<code>clock/</code>	timing utilities
<code>geometry/</code>	coordinate conversion helpers
<code>triggers/</code>	hardware trigger devices (EEG/MEG synchronization)

User code imports primarily `control` and `stimuli`. The `io` package is accessed indirectly through fields of the `Experiment` struct; direct use of `io` is necessary only when implementing custom stimulus types.

Experiment Lifecycle

Every experiment is built around a single `*control.Experiment` value that owns the SDL window and renderer, the default font, the audio device, keyboard and mouse handlers, and the output data file. The typical entry point is:

```
exp := control.NewExperimentFromFlags("My Experiment", control.Black, control.White, 32)
defer exp.End()
```

`NewExperimentFromFlags` parses two optional command-line flags (`-d` for a windowed 1024×768 developer mode, and `-s N` to set the subject ID), initializes SDL and the audio subsystem, opens the display window, and creates the data file. `defer exp.End()` ensures that the data file is flushed, fonts are released, and SDL is shut down in all exit paths, including panics. When the dialog-based participant-information collection described below is used, a lower-level constructor (`NewExperiment`) and an explicit `Initialize()` call replace this single-function entry point.

All rendering and event handling must occur on the thread that created the SDL window. `exp.Run(func() error { ... })` transfers execution to that thread and wraps the user callback in a panic/recover guard. Pressing the Escape key or closing the window triggers an internal panic with a sentinel value; `exp.Run` catches this, converts it to the `EndLoop` error, saves the data file, and returns cleanly. This mechanism means that individual calls within the trial loop do not need explicit error checking for the common case of the participant aborting the session.

Visual Stimuli

All visual stimuli implement the `VisualStimulus` interface, which requires `Draw(screen)`, `Present(screen, clear, update)`, `GetPosition()`, and `SetPosition()`. The `Stimulus` interface additionally requires `Preload()` and `Unload()` for lifecycle management.

GPU textures are allocated lazily on the first `Draw` call. For timing-sensitive code, explicit preloading via `stimuli.PreloadVisualOnScreen(screen, stim)` forces texture upload before the critical section begins.

The library provides the following visual stimulus types:

- **Text:** `TextLine` (single rendered line) and `TextBox` (word-wrapped text in a bounding rectangle). Both accept any `ttf.Font`; a default Inconsolata font is embedded in the binary.
- **Geometric shapes:** `FixCross`, `Circle`, `Rectangle`, `Line`.
- **Images:** `Picture` (loaded from file or embedded bytes). Supported formats include PNG, JPEG, and BMP.
- **Gabor patch:** `GaborPatch` — a static sinusoidal grating multiplied by a Gaussian envelope. Parameters: sigma, orientation (degrees), spatial wavelength (pixels), phase, aspect ratio, and background color.
- **Drifting grating:** `PresentMovingGrating` renders a sinusoidal grating of specified spatial frequency (cycles per pixel), temporal frequency (Hz), contrast, and mean luminance in a rectangular aperture.
- **Drifting Gabor patch:** `PresentMovingGabor` renders a drifting sinusoidal grating windowed by a Gaussian envelope with alpha-blended edges. Orientation, spatial frequency, temporal frequency, contrast, and mean luminance are all configurable.

- **Random-dot cloud:** `DotCloud` (static) and `PresentMovingDotCloud` (animated VSYNC-locked loop). In the animated version, each dot moves at a fixed speed in its assigned direction and is respawned at a random position on the cloud boundary when it exits.
- **Random-dot stereogram:** RDS generates side-by-side stereo pairs for depth perception experiments.
- **Visual mask:** `VisualMask` fills a rectangle with randomly placed dots at a specified density.
- **Composite stimuli:** `ThermometerDisplay` (a segmented progress bar) and `ChoiceGrid` (a multiple-choice button grid accepting both mouse clicks and single-character keypresses).
- **Interactive text input:** `TextInput` renders a labeled text box and collects free-text keyboard input.
- **Video:** MPEG files (via `libmpg123`) and `.gv` (LZ4-compressed RGBA sequences) can be played frame-by-frame in a VSYNC-locked loop.

Stimulus positions are specified in a **center-origin coordinate system** where (0, 0) is the screen center and positive Y is upward. The conversion to SDL's top-left origin is handled internally.

Audio Stimuli

The audio subsystem uses SDL3's audio streaming API. The `Sound` type loads WAV files from disk or embedded bytes; `Tone` generates procedural pure tones at runtime. Both implement `AudioPlayable` and can be used in audio streams.

`Sound.PlaySegment(onset, offset, rampSec)` plays the portion of a loaded WAV between `onset` and `offset` (in seconds) with optional linear fade-in and fade-out ramps. This is useful for presenting isolated phonemes or vowels extracted from longer recordings without audible clicks at segment boundaries.

Two embedded feedback sounds — a buzzer and a ping — are included in the `assets_embed` package and accessible through `exp.Audio.PlayBuzzer()` and `exp.Audio.PlayCorrect()`.

Input Handling

`exp.Keyboard` provides both blocking and non-blocking access to keyboard events:

- `Wait()` blocks until any key is pressed.
- `WaitKey(code)` blocks until a specific key is pressed.
- `WaitKeys(codes, timeoutMS)` blocks until one of several keys is pressed, with an optional timeout.
- `WaitKeysRT(codes, timeoutMS)` additionally returns the reaction time in milliseconds measured from the moment of the call.
- `WaitKeysEventRT(codes, timeoutMS)` returns the SDL3 event timestamp in nanoseconds (`KeyboardEvent.Timestamp`, populated at hardware-interrupt time via `SDL.GetTicksNS()`) rather than a polling-time delta.
- `Check()` is a non-blocking poll.
- `Clear()` drains all pending key events from the SDL queue.

The companion method `exp.ShowNS(stim)` performs the standard clear → draw → flip sequence and captures `SDL_GetTicksNS()` immediately after `SDL_RenderPresent` returns. Because `ShowNS` and `WaitKeysEventRT` both use the same SDL nanosecond clock, reaction time from any recorded stimulus onset reduces to a single subtraction with no intervening arithmetic:

```
onset, _ := exp.ShowNS(prime)
exp.Wait(500)
exp.ShowNS(target)
key, eventTS, _ := exp.Keyboard.WaitKeysEventRT(keys, 3000)
rtFromPrime := int64(eventTS - onset) // nanoseconds
```

This is particularly useful for multi-stimulus paradigms (e.g., masked priming, SOA-based designs) where the response is collected after a sequence of stimuli and RT must be anchored to a specific onset.

`exp.Mouse` provides equivalent access to mouse button events and cursor position in center-based coordinates, including a `WaitPressEventRT` method that follows the same event-timestamp convention.

Data Collection

`exp.Data` writes to a file named `<expname>_<subjectID>_<timestamp>.xpd` in `~/goxpy_data/`. The format is plain CSV with `#`-prefixed comment lines forming a structured metadata header. The header sections are:

- `--EXPERIMENT INFO`: experiment name, command line, start time, hostname, username, OS, framework version.
- `--PARTICIPANT INFO`: key/value pairs collected by the participant-information dialog (see below), if used.
- `--SUBJECT INFO`: numeric subject ID.
- `--DISPLAY INFO`: monitor ID, name, native resolution, pixel density, refresh rate, bit depth.

`exp.AddDataVariableNames(names)` writes a column-header line. Subject ID is always prepended to each data row automatically. `exp.Data.Add(values...)` appends a row; values are converted to strings via `fmt.Sprint` and fields containing delimiters or quotes are escaped. `exp.Data.Save()` flushes the buffer to disk and can be called mid-experiment to protect data against crashes.

Experimental Design

The `design` package provides a three-level `Experiment` → `Block` → `Trial` hierarchy. Each `Trial` holds a `map[string]interface{}` of factors that can be set and retrieved by name. `Block.AddTrial(trial, copies, randomPosition)` adds one or more copies of a trial; `Block.ShuffleTrials()` randomizes trial order in place.

Randomization utilities include:

- `ShuffleList(slice)`: in-place Fisher-Yates shuffle (generic, works on any slice type).
- `RandInt(a, b)`, `RandElement(slice)`, `CoinFlip(bias)`, `RandNorm(a, b)`: scalar random sampling functions.
- `MakeMultipliedShuffledList(list, n)`: concatenates `n` shuffled copies of a list.
- `RandIntSequence(first, last)`: returns a shuffled integer range.

Between-subjects Latin-square counterbalancing is supported through `exp.AddBWSFactor(name, conditions)` and `exp.GetPermutedBWSFactorCondition(name)`, which returns the condition assigned to the current subject via a balanced Latin square based on subject ID.

Pre-Experiment Setup Dialog

`control.GetParticipantInfo(title, fields)` opens a graphical SDL dialog before the experiment window is created (Figure 1). The experimenter fills in typed text fields, toggles checkbox fields, and selects from discrete option sets; pressing Enter or clicking OK returns the collected values as a `map[string]string`. The dialog supports Tab/Shift-Tab navigation between text fields, UTF-8-aware Backspace, and Escape to cancel. All values except `subject_id` are persisted to `~/.cache/goxpyriment/last.session.json` and pre-filled on the next run.

Three field types are available. **Text fields** accept free keyboard input (e.g. subject ID, session note). **Checkbox fields** toggle a boolean value (e.g. fullscreen mode). **Select fields** (`FieldSelect`) present a horizontal row of labelled buttons; clicking a button highlights it and records the corresponding value (e.g. stimulus condition, run type). This makes it practical for the experimenter or participant to choose among a small fixed set of options without typing, which reduces setup errors when launching an experiment from a desktop shortcut or file manager.

Programs that use `GetParticipantInfo` also accept a `-headless` command-line flag. When present, the dialog is skipped entirely and field defaults (or values previously saved to the cache) are used without opening any window. This is convenient for scripted or automated runs (e.g. continuous integration checks or batch stimulus generation) where interactive input is not possible.

Pre-built field sets cover common needs:

- **ParticipantFields**: subject ID, age, gender, handedness.
- **MonitorFields**: screen width in centimeters, viewing distance, refresh rate.
- **FullscreenField**: a checkbox that governs whether the experiment runs fullscreen or in a 1024×768 window.

When `exp.Info` is set before calling `exp.Initialize()`, the collected values are automatically written to the `--PARTICIPANT INFO` section of the data file header.

Hardware Triggers

The `triggers` package provides a uniform `Trigger` interface for sending digital pulses to EEG and MEG recording systems. Implementations are provided for the DLP-I/O8 USB device (a serial-port-based 8-bit digital I/O module) and Linux parallel



Fig. 1 The `GetParticipantInfo` dialog for the `Number-Comparison` example. The *Stimulus group* row shows a `FieldSelect` widget: clicking one of the four labelled buttons (Digits, Regular, Irregular, Random) highlights it in blue and records the selection. Text and checkbox fields appear above and below the select row. All values are pre-filled from the previous session's cache.

ports. `triggers.AutoDetectDLPIO8()` searches available serial ports and returns a `NullTrigger` (a no-op implementation of the interface) if no device is found, allowing the same experiment code to run with and without a recording system attached.

Timing Architecture

VSYNC Synchronization

`screen.Update()` calls SDL's `SDL_RenderPresent`, which blocks until the display's vertical retrace when VSYNC is enabled. On a 60 Hz monitor this is approximately 16.67 ms per frame; on a 144 Hz monitor approximately 6.94 ms. Because `screen.Update()` returns only after the frame has been committed to the display pipeline, the time of return provides a lower bound on stimulus onset that is accurate to within the display's frame period. This is the standard synchronization model used by the Psychophysics Toolbox (Brainard, 1997) and Expyriment (Krause & Lindemann, 2014).

For most trial-loop code, `exp.Show(stim)` — which calls `Screen.Clear()`, `stim.Draw(screen)`, and `screen.Update()` in sequence — provides frame-accurate stimulus onset without further action from the programmer.

High-Precision Streams

For paradigms with dense, high-speed stimulus sequences — rapid serial visual presentation (RSVP), attentional-blink experiments, temporal masking — the standard `exp.Show / exp.Wait` cycle introduces jitter at two points: (1) Go's garbage collector may pause execution for several milliseconds during `exp.Wait`, and (2) `exp.Wait` sleeps in 1 ms increments and does not align wakeup to frame boundaries.

The `stimuli.PresentStreamOfImages` and `stimuli.PresentStreamOfText` functions address both issues. For the duration of the stream:

1. Garbage collection is disabled via `debug.SetGCPercent(-1)` and restored on exit.

2. The SDL event queue is drained before the first frame to remove any accumulated events.
3. Each stimulus onset is scheduled as an absolute offset from the stream start; the loop busy-waits at the sub-frame level, calling `screen.Update()` on each VSYNC until the scheduled onset is reached, then draws and flips.
4. A `TimingLog` records the target onset, actual onset, and actual offset for each stimulus element. These logs allow post-hoc verification of presentation timing for each session.

The analogous `stimuli.PlayStreamOfSounds` function implements the same approach for audio sequences.

Animated Stimuli

Three functions implement self-contained VSYNC-locked animation loops for dynamic stimuli: `PresentMovingDotCloud`, `PresentMovingGrating`, and `PresentMovingGabor`. Each disables garbage collection for the duration of the loop, drains the event queue before the first frame, and returns a `MotionResult` containing the interrupt key, mouse button (if any), and reaction time from the first frame to the response.

For drifting gratings and Gabors, all spatial parameters (luminance tables, Gaussian weights) are precomputed before the loop; only the temporal phase advances per frame, requiring minimal computation within the VSYNC interval.

Limitations of the Timing Model

The VSYNC-based timing model guarantees that stimulus changes occur no earlier than the scheduled frame boundary. It does not guarantee that a change occurs on a specific frame if the CPU or GPU is busy at that moment; a missed frame will delay onset by one frame period. The stream functions minimize this risk by keeping the per-frame computation small and disabling GC, but workloads that stress the GPU — large textures, many simultaneous draws — can still cause occasional dropped frames.

Frame-accurate timing applies to stimulus onset and offset. The `WaitKeysRT` function measures elapsed time from the call site using the system clock, which introduces polling latency. The `WaitKeysEventRT` function instead returns the SDL3 `KeyboardEvent.Timestamp` field, which is populated by the OS input subsystem at hardware-interrupt time (nanoseconds since SDL initialization). This removes polling latency from response timestamps; however, the underlying keyboard interrupt scheduling on non-real-time operating systems still introduces some jitter, typically on the order of 1–5 ms.

No formal timing validation study has yet been published for `goxpyriment`. Such a study, using a photodiode and a hardware response box to measure actual stimulus onset and response timing, is a priority for future work.

Example Experiments

The repository ships with over forty example experiments in the `examples/` directory, covering a wide range of paradigms. A brief selection illustrates the breadth of the library:

- `parity_decision/`: A simple number-parity judgment task with two conditions, data logging, and a between-subjects mapping counterbalanced via Latin square. This is the recommended starting point for new users.
- `Attentional-Blink/`: A two-target RSVP task using `PresentStreamOfText` with dense 100 ms SOAs.
- `Subliminal-Priming/`: A masked priming paradigm using `PresentStreamOfImages` with backward masking; demonstrates per-stimulus timing log analysis.
- `Stroop_task/`: A classic color-word interference task with audio feedback via `exp.Audio.PlayBuzzer()`.
- `retinotopy/`: A full retinotopic mapping protocol (rotating wedge, expanding ring, sweeping bars) for fMRI, with trigger output and monitor calibration via `GetParticipantInfo`.
- `Multiple-Object-Tracking/`: A moving-dot attention task using `PresentMovingDotCloud`.
- `random-dot-stereogram/`: Cyclopean depth perception using RDS.
- `Contrast-Detection-QUEST/`: Adaptive threshold estimation using the QUEST algorithm with `PresentMovingGabor`.

Minimal Working Example

The following code implements a two-alternative parity judgment task in approximately 50 lines:

```
package main

import (
    "log"
    "github.com/chrplr/goxpyriment/control"
    "github.com/chrplr/goxpyriment/design"
    "github.com/chrplr/goxpyriment/stimuli"
)

func main() {
    exp := control.NewExperimentFromFlags("Parity", control.Black, control.White, 48)
    defer exp.End()

    exp.AddDataVariableNames([]string{"number", "parity", "key", "rt_ms", "correct"})

    numbers := []struct {
        n      string
        string
    }
```

```

    parity string
  }{
    {"1", "odd"}, {"2", "even"}, {"3", "odd"}, {"4", "even"},
    {"5", "odd"}, {"6", "even"}, {"7", "odd"}, {"8", "even"},
  }
  trials := design.MakeMultipliedShuffledList(numbers, 10)
  keys := []control.Keycode{control.K_F, control.K_J}
  fixation := stimuli.NewFixCross(20, 2, control.White)

  err := exp.Run(func() error {
    exp.ShowInstructions("F = ODD    J = EVEN\n\nPress SPACE to start.")
    stimuli.PreloadVisualOnScreen(exp.Screen, fixation)

    for _, t := range trials {
      exp.Show(fixation)
      exp.Wait(500)
      stim := stimuli.NewTextLine(t.n, 0, 0, control.White)
      exp.Show(stim)
      key, rt, _ := exp.Keyboard.WaitKeysRT(keys, 3000)
      correct := (t.parity == "odd" && key == control.K_F) ||
        (t.parity == "even" && key == control.K_J)
      exp.Data.Add(t.n, t.parity, key, rt, correct)
      stim.Unload()
      exp.Blank(500)
    }
    return control.EndLoop
  })
  if err != nil && !control.IsEndLoop(err) {
    log.Fatalf("error: %v", err)
  }
}

```

Comparison with Existing Tools

Table 1 summarizes key properties of `goxpyriment` alongside four widely used experiment frameworks.

Table 1. Comparison of selected experiment frameworks.

Property	goxpyriment	Expyriment	PsychoPy	jsPsych
Language	Go	Python	Python	JavaScript
Execution	Native binary	Interpreter	Interpreter	Browser

Property	goxpyriment	Expyriment	PsychoPy	jsPsych
Deployment	Single binary	pip/conda env	Standalone app or pip	Node.js or CDN
GC control	Explicit disable	No	No	No
VSYNC sync	Yes (SDL3)	Yes (Pygame/SDL2)	Yes (Pyglet/PTB)	Limited (rAF)
Hardware triggers	Yes (DLP-IO8, parallel)	Yes (parallel, serial)	Yes	No
Stimulus set	Moderate	Moderate	Extensive	Moderate
GUI builder	No	No	Yes (Builder)	No
Online deployment	No	No	Partial (PsychoJS)	Yes
Reference	This paper	Krause & Lindemann, 2014	Peirce et al., 2019	de Leeuw, 2015

goxpyriment most closely resembles Expyriment in its API design: a central experiment object, a **present()**-based stimulus API, a structured trial-loop model, and output to **.xpd**-format files with metadata headers. The primary technical differences are the use of Go rather than Python (providing native compilation, explicit GC control, and single-binary deployment) and the use of SDL3 rather than the older Pygame/SDL2 stack.

PsychoPy offers a substantially larger stimulus set and a graphical Builder environment that allows experimenters without programming experience to construct experiments; **goxpyriment** has neither of these. However, PsychoPy’s greater complexity also makes deployment and version management more challenging.

jsPsych enables large-scale online data collection through standard web browsers, which **goxpyriment** does not support. Browser-based presentation is subject to timing variability that is not present in native desktop applications (Bridges et al., 2020; Anwyl-Irvine et al., 2021).

The MATLAB Psychophysics Toolbox (PTB) offers the most mature timing validation in the field, with documented sub-frame onset precision on calibrated hardware (Kleiner et al., 2007). **goxpyriment** has not yet been subjected to comparable formal validation.

Timing

: TODO: check timing with photodiode, oscilloscope, ... !!!

Timing Verification Suite

The repository ships with a dedicated timing verification suite, **tests/Timing-Tests/**, that can be used to characterize the timing properties of

`goxpyriment` on a given laboratory computer before running participants. The suite is a separate Go module located under `tests/` at the repository root and is invoked as

```
go run tests/Timing-Tests/main.go -test <name> [flags]
```

Eight sub-tests are available, selected with the `-test` flag:

jitter Flips a gray screen continuously for a configurable duration (default 10 s) and records the wall-clock interval between consecutive `SDL_RenderPresent` returns. No external equipment is needed. Prints a summary including mean frame interval, SD, min/max, and the number of intervals deviating more than 0.5 ms and 1.0 ms from the mean. The measured mean is reported as an estimated refresh rate and recommended as input to the `-hz` flag of the subsequent visual tests.

frames Alternates between a dark and a bright full-screen fill for a configurable number of frames per phase (`-frames-per-phase`, default 2) over `-cycles` complete cycles. A trigger pulse is sent on a DLP-IO8-G output pin at every dark→bright transition. A photodiode attached to the bright region of the display should show a square wave whose period matches `frames-per-phase` × T_{frame} ; aligning the trigger and photodiode channels on an oscilloscope reveals the USB-serial trigger delay ($\approx 1\text{--}2$ ms).

flash Presents a single bright frame every `-isi-frames` dark frames. Use this to verify that the OS/driver combination can reliably present a one-frame stimulus: if the photodiode pulse width equals one frame period (≈ 16.7 ms at 60 Hz), single-frame precision is confirmed; double-width pulses indicate a double-buffering defect.

av Pairs a bright flash with a pure sine tone on each trial, with a configurable visual-to-audio SOA. Connecting the audio line-out and a photodiode to two oscilloscope channels yields the actual acoustic-onset latency (SDL `PutData` call to speaker), which is the quantity needed to align audio and visual events in sub-frame experiments.

square Drives a square wave on a DLP-IO8-G pin for a configurable duration, period, and duty cycle, requiring no display. Use this to characterize the USB-serial trigger jitter in isolation before attributing display-trigger misalignment to the display.

sound Plays a long isochronous tone stream (default: 300 tones × 50 ms on / 450 ms ISI ≈ 2.5 min) and logs the onset time of each tone relative to the scheduled target. Reports onset error and inter-onset interval statistics; a cumulative drift in onset error indicates that the wall clock and the audio clock diverge over time. An optional DLP-IO8-G trigger per tone, combined with an oscilloscope and the audio line-out, yields the software-to-DAC latency for each tone.

rt Measures keyboard reaction time using SDL3 hardware event timestamps. Each trial presents a one-frame white flash; $RT = \text{KeyboardEvent.Timestamp} - \text{FlipNS}()$, where both times come from the same `SDL.GetTicksNS()` clock. This removes polling latency from the response side; the residual jitter reflects OS keyboard scheduling (typically < 2 ms on Linux).

drain Measures audio pipeline latency without any external equipment. For each of five fixed tone durations (25, 50, 100, 200, 500 ms), it repeats `-drain-reps` trials. Each trial calls `Play()`, then polls `stream.Queued()` in a 0.5 ms sleep loop until the SDL audio stream is empty. The elapsed time from `Play()` to drain completion equals the nominal tone duration plus the hardware-buffer pipeline latency. The SD across reps

measures trial-to-trial jitter in the audio scheduler. This test requires no microphone or oscilloscope and is suitable for a quick pre-experiment sanity check on any machine.

All sub-tests accept a `-hz` flag (default 60.0) that sets the expected display refresh rate for computing frame-interval targets; the recommended workflow is to run `jitter` first and pass the measured value to the subsequent visual tests. A `-warmup` flag (default 10 frames) discards the initial frames from statistics to avoid GPU pipeline warm-up bias. Each run writes an `.xpd` file to `~/goxpy_data/` that can be loaded into Python for further analysis.

Why Go?

The choice of Go as the implementation language for `goxpyriment` warrants explicit justification, as the behavioral research community is more used to use Python-based tools.

Go was designed with a deliberate emphasis on simplicity and predictability and has far less “moving parts” than Python. Here are features that make Go a sensible choice for an experimental framework.

Static typing and compile-time safety. Go is statically typed: type errors, missing methods, and incorrect function signatures are caught at compile time rather than at runtime. In Python, dynamic typing allows such errors to surface only during an ongoing experiment, potentially corrupting a data-collection session mid-run. Compile-time feedback provides an important safety net.

Native concurrency. Go provides lightweight concurrency through goroutines and channels. This is relevant for behavioral experiments that must coordinate multiple concurrent processes — stimulus rendering, response monitoring, hardware trigger dispatch, and audio playback — with precise timing. Python’s concurrency primitives, including the Global Interpreter Lock and the `asyncio` framework, impose architectural constraints that complicate low-latency multi-threaded coordination.

Explicit garbage-collection control. Go exposes a runtime API for suspending and resuming the garbage collector (`debug.SetGCPercent`). `goxpyriment` uses this mechanism in its VSYNC-locked stimulus presentation loops: GC is disabled for the duration of a timing-critical sequence and restored immediately afterward (see Timing Architecture). Python provides no equivalent facility.

Suitability for AI-assisted development. Researchers in the behavioral sciences are not always experienced programmers, and AI-powered coding assistants have become a practical tool for writing and maintaining experiment code. Go’s properties appear to be particularly favorable in this context. Because the language has few synonymous constructs and a strict type system, the solution space explored by a language model is narrower and more constrained than in Python. Errors introduced by the model — such as calling a non-existent method or passing an argument of the wrong type — are caught at compile time before any code is executed. This makes Go an attractive choice for a library intended to be used by researchers who rely on AI coding assistants to bridge gaps in their programming experience.

Single-binary deployment. A Go program compiles to a single statically linked executable with no external runtime dependencies. Distributing an experiment requires copying one file; there is no need to manage virtual environments, package versions, or interpreter installations across laboratory computers. This substantially reduces the operational overhead of running experiments on multiple machines or sharing materials with collaborators.

Taken together, these properties make Go well suited to the requirements of a low-level behavioral experiment framework: predictable runtime behavior, precise timing control, simple deployment, and a type system that catches errors before they can affect data quality.

Limitations and Future Directions

`goxpyriment` is currently at the beta-stage. Although we believe that it can already be used to implement real experiments in the lab, users should (as always) very carefully check their behavior, for example with a Black Box Tool Kit (see [bbtvk3 software](#)).

Several limitations should be noted:

No formal timing validation. The timing architecture is theoretically sound — VSYNC synchronization and GC suspension are established techniques — but no published photodiode or hardware-response-box measurements of actual onset precision exist for this library as of this writing. The `tests/Timing-Tests/` suite (Section 2) provides the tools needed to perform such measurements on a given laboratory computer, and a formal validation study using a photodiode and a hardware response box is a priority for future work. Users requiring certified timing precision should treat claims with appropriate caution until such benchmarks are published.

No graphical experiment builder. All experiments are written in Go code. This requires programming experience that not all researchers have. However, the library is designed with a simple, consistent API that lends itself well to use with AI coding assistants.

Limited stimulus set relative to PsychoPy. The current library covers the stimuli most commonly needed in cognitive psychology and cognitive neuroscience — text, shapes, images, gratings, Gabors, dot clouds, stereograms. It does not yet include, for example, movie stimuli from standard video formats (only the custom `.gv` format and MPEG), 3D rendering, or stereoscopic displays.

Desktop-only. There is no support for online data collection via a web browser. The support of web assembly (wasm) by `go-sld3` is still too premature, but when this is corrected, one will be able to compile to a wasm binray that will target browsers.

No eye-tracker support. `goxpyriment` currently provides no interface to eye-tracking hardware. Eye-tracking is widely used in cognitive psychology, reading research, and clinical assessment, so this is a notable gap. We plan to address it in a future release, with initial support targeting the SR Research EyeLink 1000 — one of the most common laboratory eye-trackers — via its C SDK. A clean abstraction layer will be designed from the outset so that additional devices (e.g. Tobii, SMI) can be integrated without changes to experiment code.

AI assisted coding

AI excels at programming, but human guidance is still necessary as exemplified by Fig. 2 (generated in Feb. 2026)

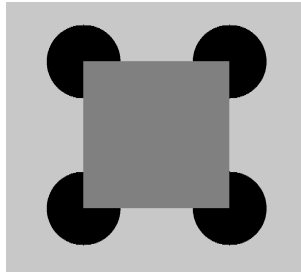


Fig. 2 Asked to write a program displaying Kanizsa’s illusory rectangle, an AI produced the result above, explaining: “I have changed the Kanizsa example to make things robust and more visible [...] A darker gray rectangle is drawn at the center, clearly visible against the light-gray background.”

Availability and Documentation

goxpyriment is freely available under the GNU General Public License v3 at:

<https://github.com/chrplr/goxpyriment>

Prerequisites. Go 1.25 or later. No other runtime dependencies are required; the SDL3, SDL3_ttf, and libmpg123 binaries are bundled via the `go-sdl3/bin` packages and are loaded at runtime.

Building experiments. Clone the repository, navigate to any example directory, and run `go run .`. To compile a standalone binary for distribution, run `go build .` in the experiment directory.

Documentation. The repository includes:

- `docs/GettingStarted.md`: a tutorial for researchers familiar with Expyriment or PsychoPy.
- `docs/UserManual.md`: a concept guide covering all major subsystems.
- `docs/API.md`: a complete public API reference organized by package.

An online documentation site is published at <https://chrplr.github.io/goxpyriment>.

Author Contributions

Christophe Pallier: Conceptualization, Software (original conception and architecture, all design decisions, integration and testing), Writing — original draft (conceptualization and framing), Writing — review and editing.

Claude Sonnet 4.6 (Anthropic): Software (substantial implementation of multiple modules, including `control/getinfo.go`, `io/data.go` extensions, `stimuli/moving_grating.go`, `stimuli/moving_dotcloud.go`, and portions of `control/experiment.go`; all co-authored `.go` files carry an explicit `// Co-authored by Claude Sonnet 4.6` header comment), Writing — original draft (the paper text was drafted collaboratively in an interactive session).

Note on AI authorship: consistent with current editorial norms (Nature Portfolio, 2023; Springer Nature, 2023), Claude Sonnet 4.6 is listed under Author Contributions rather than as a named author, because authorship requires the capacity to take accountability for the work. Christophe Pallier takes full responsibility for all content, including the AI-assisted portions.

Open Practices Statement

The source code of `goxpyriment` is available under the GNU General Public License v3 at <https://github.com/chrplr/goxpyriment>. All example experiments described in this article are included in the repository. No empirical data are reported in this article.

References

- Anwyl-Irvine, A. L., Massonnié, J., Flitton, A., Kirkham, N., & Evershed, J. K. (2021). Gorilla in our midst: An online behavioral experiment builder. *Behavior Research Methods*, 52(1), 388–407. <https://doi.org/10.3758/s13428-019-01237-x>
- Brainard, D. H. (1997). The Psychophysics Toolbox. *Spatial Vision*, 10(4), 433–436. <https://doi.org/10.1163/156856897X00357>
- Bridges, D., Pitiot, A., MacAskill, M. R., & Peirce, J. W. (2020). The timing mega-study: Comparing a range of experiment generators, both lab-based and online. *PeerJ*, 8, e9414. <https://doi.org/10.7717/peerj.9414>
- de Leeuw, J. R. (2015). jsPsych: A JavaScript library for creating behavioral experiments in a Web browser. *Behavior Research Methods*, 47(1), 1–12. <https://doi.org/10.3758/s13428-014-0458-y>
- de Leeuw, J. R., Gilbert, R. A., & Luchterhandt, B. (2023). jsPsych: Enabling an open-source collaborative ecosystem of behavioral experiments. *Journal of Open Source Software*, 8(85), 5351. <https://doi.org/10.21105/joss.05351>
- Donovan, A. A. A., & Kernighan, B. W. (2016). *The Go programming language*. Addison-Wesley.
- Henninger, F., Shevchenko, Y., Meijer, R. R., Inzlicht, M., & Hilbig, B. E. (2022). lab.js: A free, open, online study builder. *Behavior Research Methods*, 54(2), 556–573. <https://doi.org/10.3758/s13428-019-01283-5>
- Kleiner, M., Brainard, D. H., & Pelli, D. (2007). What’s new in Psychtoolbox-3? *Perception*, 36(ECVP Abstract Supplement), 14.

Krause, F., & Lindemann, O. (2014). Expyriment: A Python library for cognitive and neuroscientific experiments. *Behavior Research Methods*, 46(2), 416–428. <https://doi.org/10.3758/s13428-013-0390-6>

Peirce, J. W., Gray, J. R., Simpson, S., MacAskill, M. R., Höchenberger, R., Sogo, H., Kastman, E., & Lindeløv, J. K. (2019). PsychoPy2: Experiments in behavior made easy. *Behavior Research Methods*, 51(1), 195–203. <https://doi.org/10.3758/s13428-018-01193-y>

Nature Portfolio. (2023). *Tools and technologies: Artificial intelligence*. <https://www.nature.com/nature-portfolio/editorial-policies/ai>

Springer Nature. (2023). *Artificial intelligence (AI) policy for authors*. <https://www.springernature.com/gp/policies/editorial-policies>

Zyko0. (2024). *go-sdl3: Go bindings for SDL3*. GitHub. <https://github.com/Zyko0/go-sdl3>

OS Security Warnings When Distributing Compiled Experiments

A key advantage of goxpyriment is that a compiled experiment is a single self-contained binary that can be copied to any laboratory or participant computer without installing Go or any runtime library. In practice, both macOS and Windows apply security mechanisms that warn users — or actively block execution — the first time a binary downloaded from the internet or copied from an external drive is run. Researchers who plan to distribute compiled experiments to colleagues or participant computers should be aware of these mechanisms.

macOS: GateKeeper

Apple’s GateKeeper evaluates applications against three tiers of trust:

1. **Unsigned** — no signature. GateKeeper displays a dialog stating the binary “is damaged and can’t be opened.” This message is misleading: the file is not corrupted; it lacks a signature.
2. **Ad-hoc signed** — signed with `codesign --sign -` at no cost, without an Apple Developer account. GateKeeper shows a one-time warning on first launch but allows the user to proceed.
3. **Apple-notarized** — signed and notarized through Apple’s service (requires a paid Apple Developer Program membership, \$99/year). No warning is shown.

Recommended practice for researchers distributing experiments: apply ad-hoc signing before distribution, which changes the user experience from a blocking “damaged” error to a one-time confirmation prompt.

```
codesign --sign - --force --deep my_experiment
```

For recipients running an unsigned binary: right-click (or Control-click) the binary and select “Open”, then confirm in the dialog. This is required only once per binary. Alternatively: System Settings → Privacy & Security → “Open Anyway”.

For command-line binaries transferred via USB or network share (which may have the quarantine attribute set):

```
xattr -d com.apple.quarantine my_experiment  
chmod +x my_experiment
```

For a complete packaging guide and a shell script that creates a distributable `.app` bundle with ad-hoc signing, see: <https://chrplr.github.io/note-about-macos-unsigned-apps/>

Windows: SmartScreen

Windows SmartScreen (part of Microsoft Defender) evaluates executables using a reputation system. An executable without sufficient download history — typical for laboratory software not distributed through the Microsoft Store — triggers a blue dialog: “Windows protected your PC. Microsoft Defender SmartScreen prevented an unrecognized app from starting.”

To run the experiment: click “More info” in the dialog, then “Run anyway”. This confirmation is required once per executable.

Obtaining an Extended Validation (EV) code-signing certificate from a commercial certificate authority would suppress this warning automatically but involves ongoing cost. For internal laboratory use, the “Run anyway” procedure is the standard workaround.

Note on Timing

Security prompts on both platforms are presented by the OS before the experiment binary is launched. They do not affect the timing of stimulus presentation within the experiment itself.